Federated Fine-Tuning on Heterogeneous LoRAs With Error-Compensated Aggregation

Wanyi Ning, Jingyu Wang, *Senior Member, IEEE*, Qi Qi, *Senior Member, IEEE*, Haifeng Sun, *Senior Member, IEEE*, Daixuan Cheng, Cong Liu, Lei Zhang, Zirui Zhuang, *Member, IEEE*, and Jianxin Liao

***Abstract*—Federated learning (FL) has recently been applied to the parameter-efficient fine-tuning (PEFT) of large language models (LLMs). While promising, client resource heterogeneity has imposed the challenge of the "bucket effect" to FL, where model configuration must cater to the client with the fewest resources. To tackle this issue, heterogeneous low-rank adaptation (LoRA) has recently emerged in FL, which enables clients to do local fine-tuning with different LoRA ranks. However, existing works in this area typically adopt zero-padding, stacking, or singular value decomposition (SVD) for LoRA aggregation, which often incur precision loss or significant overhead, limiting their practicality. In this article, we propose ECLoRA, a novel method for federated fine-tuning with heterogeneous LoRA settings across clients. ECLoRA employs randomized SVD (RSVD) to dramatically reduce aggregation overhead while introducing an error compensation (EC) mechanism that incorporates the decomposition error from previous rounds to improve aggregation precision. Extensive experiments on four widely used foundation models across six public tasks demonstrate the effectiveness of ECLoRA. Specifically, ECLoRA is: (1) *accurate*, significantly improving the final model performance; (2) *fast*, accelerating convergence with an average speedup of 1.54× to 3.01×; and (3) *practical*, reducing aggregation time by approximately 40× compared to classical SVD.**

***Index Terms*—Federated learning (FL), low-rank adaptation (LoRA) fine-tuning.**

NOMENCLATURE

Symbol	Definition
W	Full-rank model weights.
ΔW	Full-rank model updates.
θ	LoRA weights with two low-rank matrices A and B.
f	Loss function.
∇f	True gradient of loss function.
g	Statistic gradient based on minibatch.
N	Total number of clients.
$\mathcal{D}_i$	Local dataset of the ith client.
r_i	Local LoRA rank of the ith client.
$\mathcal{S}^t$	Selected client set in round t.
T	Number of communication rounds.
γ	Participation rate in FL.
$\mathcal{R}$	Involved ranks set in FL with heterogeneous LoRAs.
e	Decomposition error in ECLoRA.
β	Compensation factor in ECLoRA.
$\Delta\hat{W}$	Error-compensated full-rank model updates in ECLoRA.
Rec(·)	Function of reconstructing LoRA weights into full rank.
RSVD(·)	Function of RSVD.

Received 27 November 2024; revised 29 April 2025; accepted 26 June 2025. Date of publication 17 July 2025; date of current version 9 October 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2024YFE0200800; in part by the National Natural Science Foundation of China under Grant 62321001, Grant 62471055, Grant U23B2001, Grant 62101064, Grant 62171057, and Grant 62201072; in part by the High-Quality Development Project of the MIIT under Grant 2440STCZB2584; in part by the Ministry of Education and China Mobile Joint Fund under Grant MCM20200202 and Grant MCM20180101; and in part by the Fundamental Research Funds for the Central Universities under Grant 2024PTB-004. *(Corresponding authors: Zirui Zhuang; Qi Qi.)*

Wanyi Ning, Jingyu Wang, Qi Qi, Haifeng Sun, Daixuan Cheng, Zirui Zhuang, and Jianxin Liao are with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China, and also with E-Byte.COM, Beijing 100191, China (e-mail: ningwanyi@bupt.edu.cn; wangjingyu@bupt.edu.cn; qiqi8266@bupt.edu.cn; hfsun@bupt.edu.cn; daixuancheng6@gmail.com; zhuangzirui@bupt.edu.cn; liaojx@bupt.edu.cn).

Cong Liu is with China Mobile Research Institute, Beijing 100053, China (e-mail: liucong@chinamobile.com).

Lei Zhang is with China United Network Communications Corporation, Beijing 100032, China (e-mail: zhangl83@chinaunicom.cn).

Digital Object Identifier 10.1109/TNNLS.2025.3586545

I. INTRODUCTION

LARGE language models (LLMs) have demonstrated exceptional performance across a range of tasks, such as chatbots, virtual assistants, search engines, and healthcare [1], [2], [3], [4], [5]. However, fine-tuning all the parameters of pretrained LLMs for specific downstream tasks demands substantial computational resources. To address this, various parameter-efficient fine-tuning (PEFT) methods have been developed, with low-rank adaptation (LoRA) being one of the most widely adopted [6], [7], [8], [9], [10]. Instead of fine-tuning the entire model, LoRA updates only two small matrices while keeping the pretrained backbone frozen [9], [10]. This significantly cuts down on the number of trainable parameters, reducing both computational and communication costs.

Given that the downstream data for fine-tuning LLMs are often generated or collected from individuals in a distributed manner, there arises a need for effective training methods that can leverage this data while preserving privacy. In this context, federated learning (FL) serves as a distributed learning paradigm, enabling multiple individual clients to jointly train a global model through frequent parameter exchanges without sharing their data [11], [12], [13], [14], [15]. While it is natural to apply LoRA tuning to FL, there is a limitation that all clients must configure their LoRA with the same rank to ensure dimensional consistency during aggregation [16]. However, in FL, the available resources usually vary among clients, especially in cross-device scenarios. The limitation leads to the "bucket effect" [17], [18], converging to the use of the

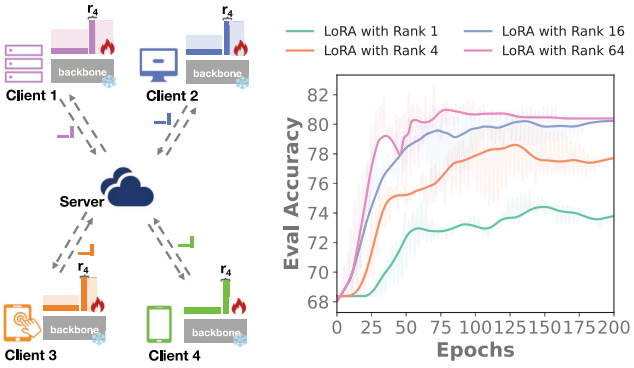

Fig. 1. *Left:* All clients must configure their LoRA with the same minimal rank as Client 4 to ensure dimensional consistency during aggregation, leading to "bucket effect." *Right:* Learning curves of fine-tuning the RoBERTa-base model on the MRPC dataset using LoRA in FL. Fine-tuning with the minimal rank limits the model convergence.

smallest involved LoRA rank for all clients while not fully utilizing the computing resources of clients.

As shown in Fig. 1, the other three clients who could have set a larger rank are forced to use the minimal rank as in Client 4. Since the rank of LoRA represents the complexity and capacity of the model's adaptation to the specific task [9], [19], the rank limitation will restrict the overall performance of the global model in FL. The right learning curves in Fig. 1 illustrate this restriction where the convergence of the global model in FL significantly drops with a decrease in LoRA rank. Therefore, it is desired to develop a method that allows federated fine-tuning with heterogeneous LoRA ranks, accelerating the model convergence and reducing the required communication rounds.

Although there have been several works for federated fine-tuning on heterogeneous LoRAs, they are hardly practical due to their inaccurate or time-consuming aggregation schemes [17], [18], [18], [20]. For example, HETLORA uses a simple and direct aggregation scheme with zero-padding and truncation, while it does not fully utilize the convergence advantage with different ranks and is not accurate enough [20]. FLoRA stacks heterogeneous LoRA modules for aggregation, but the need for reinitialization in each round slows down its convergence [18]. FlexLoRA reconstructs the full-rank update matrix in the server and applies singular value decomposition (SVD) [21] on the full-rank update matrix to obtain the updated global LoRA weights [17]. However, given the hundred billion size of the model, the computational cost of SVD is enormous and very time-consuming.

Can we enable federated fine-tuning on heterogeneous LoRAs to effectively improve model convergence and performance with tolerable cost? The answer is "yes." In this work, we propose a novel method—error-compensated LoRA tuning, dubbed as ECLoRA,[1] for federated fine-tuning on heterogeneous LoRAs. To keep the dimension consistency in aggregation, ECLoRA reconstructs the full-rank update matrix on the server and decomposes it through randomized SVD (RSVD) for broadcasting. Compared to classical SVD, RSVD significantly reduces computational overhead and improves efficiency with a small loss in precision [22]. Besides, ECLoRA also introduces an EC error compensation (EC) mechanism to further improve the model convergence. Specifically, the server stores the decomposition error of RSVD by conducting a subtraction between the original full-rank update matrix and the decomposed one. In each round, the server adds the stored previous-round error into the current-round update matrix and then decomposes the compensated update matrix. In this way, ECLoRA can effectively make up for the errors caused by RSVD decomposition and improve the model convergence and performance at a small computational cost.

For evaluation, we perform extensive experiments on four widely used foundation models with different sizes (RoBERTa-base [23], RoBERTa-large [23], DeBERTa-XXL [24], and LLaMA-3.1-8B [25]) for six public downstream natural language processing tasks (MRPC [26], CoLA [26], QNLI [26], RTE [26], SQuAD [27], and CoNLL-2003 [28]). The results reveal the following advantages of ECLoRA.

1) *Accurate:* It fully utilizes the advantage of heterogeneous LoRAs, significantly improving the model performance, such as 4.9%+ on $\text{RoBERTa}_{\text{base}}$ MRPC and 3.2%+ on $\text{LLaMA-3.1}_{\text{8B}}$ RTE.
2) *Fast:* It significantly accelerates model convergence, such as with 1.25×-4.16× speedup on $\text{RoBERTa}_{\text{large}}$, reducing the number of communication rounds required to reach the target accuracy, which is crucially important in FL.
3) *Practical:* Compared to FlexLoRA using classical SVD, it significantly reduces the aggregation time by 30× to 40× while keeping the superior convergence, which can be easily applied in practice.

The rest of this article is organized as follows. In Section II, we present the preliminaries, introducing LoRA and the resource heterogeneity in FL, along with some existing related works. Then, it follows our proposed error-compensated method ECLoRA in Section III. Experiment results are presented to validate the performance of our method in Section IV. Finally, this article is concluded in Section V. For convenience, the main notations used in this article are listed in Nomenclature.

II. Preliminaries

A. LoRA and Rank

LoRA is a PEFT method designed to reduce the computational and memory overhead of adapting LLMs [9], [10]. The core principle of LoRA involves decomposing the model updates into two low-rank matrices, which represent the adjustments to the model's pretrained weights. The weight update mechanism in LoRA can be expressed as

$$W' = W + \Delta W = W + BA \tag{1}$$

where $W \in \mathbb{R}^{d\times m}$ represents the pretrained weights and $W' \in \mathbb{R}^{d\times m}$ are the updated weights. The update term ΔW is decomposed into $A \in \mathbb{R}^{r\times m}$ and $B \in \mathbb{R}^{d\times r}$, where r is the rank of the low-rank matrices, typically much smaller than d and m. By optimizing the smaller matrices A and B rather

[1]Our code is available online at https://github.com/ningwanyi/ECLoRA

TABLE I
COMPARISON OF TRAINABLE PARAMETERS, GPU MEMORY, SPEED, AND FL ACCURACY ACROSS DIFFERENT LORA RANKS ON ROBERTA$_{\text{BASE}}$ MRPC

Rank	Trainable Params	GPU Memory (MB)	Throughput (it/s)	FL Accuracy
1	592,898	8,094	4.29	74.5
4	2,362,370	8,116	4.13	80.1
64	37,751,810	8,570	3.83	85.0
128	75,500,546	9,618	3.75	88.0

than the full-rank matrix ΔW, LoRA significantly reduces the number of trainable parameters. This reduction leads to lower memory consumption and computational requirements, making the fine-tuning process more efficient on resource-constrained devices.

The choice of rank r is crucial for balancing the tradeoff between model performance and efficiency. A higher rank r allows the model to capture more detailed information and complex patterns in the data, potentially improving model performance and accuracy. However, this also increases the computation and memory demands. Conversely, a lower rank reduces the computation burden but may limit the model's capacity to learn intricate patterns, possibly leading to a decrease in accuracy. As shown in Table I, increasing the LoRA rank improves FL accuracy, reaching up to 88.0% with rank 128, but also requires more computation resources, indicating a tradeoff in performance and efficiency.

B. Resource Heterogeneity in FL

One of the main features of FL is that clients may differ in their computational resources. Clients with limited computation resources can restrict the model configuration, often leading to suboptimal utilization of the available resources across all clients [11], [29], [30]. Consider an FL setting with N clients, where each client $i \in \{1, 2, \ldots, N\}$ has its own local computational capacity and can handle an LoRA rank $r_i \in \{r_1, r_2, \ldots, r_N\}$. However, traditional FL requires all client models to have the same architecture for aggregation. Therefore, the actual LoRA model rank has to be adapted to accommodate the client with the minimum resources, denoted as $r_{\min}$, leading to underutilization of more capable clients, which is also called "bucket effect" [17], [18], [20].

The "bucket effect" limitation hampers the potential benefits of clients with higher computational capacities. Clients capable of handling higher ranks $r_i > r_{\min}$ are forced to downscale their updates, resulting in a loss of finer-grained adjustments. Ideally, each client should operate at its maximum capability r_i, but the need for a common rank during aggregation limits this flexibility. As a result, the overall fine-tuning process is dominated by the least capable client, leading to inefficiency and slower convergence.

C. Related Works

PEFT aims to reduce the number of trainable parameters by selectively updating only a subset of the model's parameters during fine-tuning, such as prefix-tuning [31], adapter [8], and LoRA [9], [10]. There have been a variety of efforts integrating PEFT techniques with FL. Here, we mainly focus on federated LoRA fine-tuning since LoRA is the most widely used and effective PEFT technique [9], [10]. The existing works in federated LoRA fine-tuning can be divided into two categories based on local LoRA heterogeneity: 1) one sets a homogeneous LoRA rank for all clients [16], [32], [33] and 2) the other sets the heterogeneous LoRA ranks for different clients [17], [18], [20], [34].

FedIT [16] is the first approach to combine FedAvg with LoRA for federated fine-tuning, where clients fine-tune LoRA modules with a uniform rank. Clients fine-tune the LoRA modules on a common pretrained LLM with their local data. The server receives the uploaded LoRAs from all clients and updates the global LoRA modules A and B by performing weighted averaging across all local modules A_i and B_i

$$A = \sum_{i=1}^{N} p_i A_i, \quad B = \sum_{i=1}^{N} p_i B_i. \tag{2}$$

The aggregation process of FedIT is similar to FedAvg, with the key difference being that only the LoRA modules are trained and communicated. In addition to FedIT, there are also several other works applying homogeneous LoRAs in FL [32], [33]. However, they all just apply LoRA in FL for memory reduction compared with full fine-tuning, without consideration of client resource heterogeneity.

Cho et al. [20] introduce HETLORA, which first allows clients to have different LoRA ranks in FL. This is achieved by zero-padding the local LoRA weights for aggregation and truncating the global weights to align with the local ranks during distribution. However, HETLORA inevitably loses information and causes accuracy loss due to simple zero-padding and truncation. Bai et al. [17] also explore federated fine-tuning on heterogeneous LoRAs and introduce FlexLoRA. This method reconstructs a full-rank LoRA update from individual local parameters and employs SVD for redistribution. Despite the performance advantages of FlexLoRA, its practicality is severely limited by the extremely time-consuming nature of SVD, which becomes especially prohibitive with LLMs with huge model sizes. Wang et al. [18] propose a stacking-based aggregation method, FLoRA, for federated fine-tuning on heterogeneous LoRAs. Despite its noise-free feature, it suffers from significant communication overhead due to broadcasting the entire stacked modules to all clients and experiences slower convergence because of the need to reinitialize LoRA parameters at the start of each training round. Different from the above methods, our method ECLoRA enables federated fine-tuning on heterogeneous LoRAs by utilizing RSVD for aggregation with an EC mechanism, which has much less computation overhead compared to FlexLoRA but achieves superior model performance. We present our method in detail in Section III.

III. ECLoRA

In federated fine-tuning on heterogeneous LoRA ranks, there are two main concerns: 1) how to aggregate all the local LoRAs with different ranks and 2) how to broadcast the

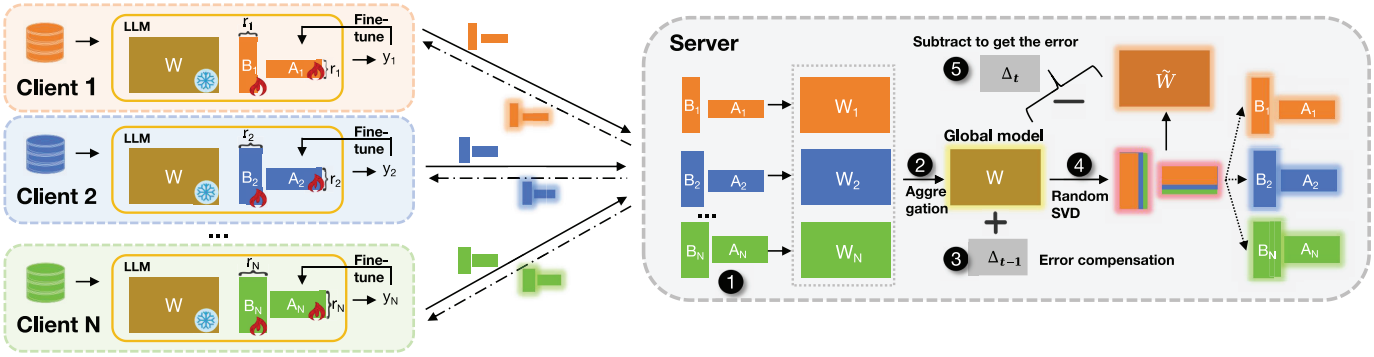

Fig. 2. Overview of ECLoRA. Each client initially sets its own rank of LoRA according to its resources. In each round, the selected clients use their local data to fine-tune the LoRA module and upload it to the server. After receiving all the local LoRAs, the server (1) reconstructs their update matrices by multiplying A and B of LoRA and (2) aggregates these local matrices to get the global one. The aggregated global matrix is (3) compensated by adding the previous-round error. Then, the compensated global matrix is (4) decomposed through RSVD into a global LoRA module with the maximal rank for flexible broadcasting. Meanwhile, the server (5) reconstructs this global LoRA module and subtracts it from the compensated global matrix to get the current-round error.

global model to all the clients. Our proposed method ECLoRA addresses these concerns through an error-compensated SVD mechanism.

We present the overview of ECLoRA in Fig. 2. In each round, the clients freeze the pretrained backbone of LLM and fine-tune the LoRA module based on their local data. The local LoRA modules are then uploaded to the server. Corresponding to concern 1), the server reconstructs the local update matrices from these LoRA modules for aggregation. Corresponding to concern 2), the aggregated global update matrix is first compensated with the previous-round decomposition error and is then decomposed through an RSVD function for the subsequent broadcasting. We next introduce the two main procedures of ECLoRA, reconstruction for aggregation and RSVD with EC, in detail.

A. Reconstruction for Aggregation

1) Reconstruction: Let $\theta_i = \{A_i, B_i\}$ be the LoRA module parameters for the ith client. Let $\text{Rec}(\theta)$ be the reconstruction function. After receiving the local LoRA modules, the server reconstructs the local update matrices ΔW_i as

$$\Delta W_i \in \mathbb{R}^{d\times m} = \text{Rec}\,(\theta_i) = B_i A_i \tag{3}$$

where d and m are the 2-D of ΔW_i, $B_i \in \mathbb{R}^{d\times r_i}$ and $A_i \in \mathbb{R}^{r_i\times m}$ are the low-rank matrices from the ith client, and r_i is the client's LoRA rank. The reconstruction function $\text{Rec}(\theta)$ maps the low-rank parameter representation θ back to the full-rank update matrix ΔW, aligning with the dimensions of the backbone weight W. This provides the correct mapping for the subsequent aggregation.

2) Aggregation: After reconstructing the local update matrices ΔW_i for each client, the server aggregates these matrices to form the global update matrix ΔW. The aggregation is performed by calculating a weighted sum of the reconstructed local updates from all participating clients $\mathcal{S}^t$ in round t. The aggregated global update matrix $\Delta W^{(t)}$ is computed as

$$\Delta W^{(t)} = \sum_{i=1}^{|\mathcal{S}^t|} \frac{|\mathcal{D}_i|}{\sum_{j=1}^{|\mathcal{S}^t|}|\mathcal{D}_j|} \Delta W_i^{(t)} \tag{4}$$

where $\mathcal{S}^t$ denotes the set of clients participating in round t and $|\mathcal{S}^t|$ is the number of participating clients. $W_i^{(t)}$ is the reconstructed local update matrix for the ith client in the current round. The term $\mathcal{D}_i$ is the local dataset of the ith client so that the clients with more data have a proportionally larger impact on the global update.

B. RSVD With EC

1) Error Compensation: To broadcast the low-rank modules to clients in the next round, we use RSVD in the server to decompose the aggregated global update matrix ΔW into the maximal rank r of the involved client ranks as

$$\text{RSVD}_r\,(\Delta W) = U\Sigma V^{\mathrm{T}} \tag{5}$$

where U, Σ, and V^T are the SVD components of ΔW. Then, the global LoRA module θ can be obtained as

$$\theta = \{A, B\} = \{V^T, U\Sigma\} \tag{6}$$

where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times m}$ are the low-rank matrices in the global LoRA module θ. However, as a dimension reduction technique, RSVD will inevitably bring errors. Our goal is to reduce the error between the low-rank matrix $\theta = \{A, B\}$ and the full-rank matrix ΔW. The error e is obtained by reconstructing the global low-rank module θ, which can be expressed as

$$e = \Delta W - \text{Rec}\,(\theta)\,. \tag{7}$$

To reduce the impact of the errors, we compensate for the current global matrix by adding the previous-round error $e^{(t)}$ before RSVD, which is expressed as

$$\begin{aligned} \Delta\hat{W}^{(t)} &= \Delta W^{(t)} + \beta e^{(t)} \\ e^{(t)} &= \Delta\hat{W}^{(t-1)} - \text{Rec}\left(\theta^{(t-1)}\right) \\ \theta^{(t)} &= \text{RSVD}_r\left(\Delta\hat{W}^{(t)}\right) \end{aligned} \tag{8}$$

where $\Delta\hat{W}^{(t)}$ is represented as the compensated global update matrix in the current round t and β is the compensation factor, which affects the importance of the compensation term. In this way, the error $e^{(t)}$ introduced by RSVD in the previous round can be compensated for in the current round, making up for the information loss caused by the decomposition.

2) Randomized SVD: The core idea of RSVD is employing random projection to compress the original high-dimensional matrix into a low-dimensional subspace, performing an exact SVD within this subspace, and subsequently projecting the result back to the original space. We illustrate the workflow of RSVD in Fig. 3, following the steps given in the following [22].

1) *Step 1:* Initially, a random projection matrix $\Omega \in \mathbb{R}^{m\times p}$ is generated, where $p \le m$ is the projecting dimension, usually set to multiples of the low rank. The aggregated matrix $\Delta\hat{W}^{(t)} \in \mathbb{R}^{d\times m}$ is then projected onto a low-dimensional subspace as
$$Y = \Delta\hat{W}^{(t)}\Omega. \tag{9}$$
This projection step reduces the original problem from dimension $d \times m$ to $d \times p$, which significantly decreases the computational cost of subsequent operations. The time complexity of this projection step is $O(dmp)$.
2) *Step 2:* We perform SVD decomposition on Y to obtain an orthonormal matrix $\text{Orth}(Y) = U_Y \sum_Y \in \mathbb{R}^{d\times p}$, which serves as a basis for the subspace. The time complexity of this step is $O(dp^2)$.
3) *Step 3:* We project the original matrix $\Delta\hat{W}^{(t)}$ onto the orthonormal basis
$$G = \text{Orth}(Y)^T \Delta\hat{W}^{(t)} \tag{10}$$
where G is a much smaller matrix compared to the original weight matrix, ensuring that the subsequent decomposition can be performed efficiently. The time complexity of this step is $O(pdm)$.
4) *Step 4:* Next, we perform a standard SVD on the smaller matrix G to obtain its approximate SVD $G = U_G \Sigma V$. The time complexity of this step is $O(mp^2)$.
5) *Step 5:* Finally, we project the left singular vectors back into the original high-dimensional space as
$$U = \text{Orth}(Y)U_G. \tag{11}$$
Thus, we obtain the approximate low-rank decomposition $\Delta\hat{W}^{(t)} \approx U\Sigma V^T$. The global LoRA parameters are then formed as
$$\theta^{(t)} = \{A, B\} = \{V^T, U\Sigma\} \tag{12}$$
where $A \in \mathbb{R}^{p\times m}$ and $B \in \mathbb{R}^{d\times p}$ are the low-rank matrices that will be broadcast to the clients. Specifically, the rank of the broadcasted LoRA modules can be adjusted flexibly as $\theta^{(t)}[: r_i]$ to match each client's local rank r_i as line 4 in Algorithm 1. The time complexity of this step is $O(dp^2)$.

3) Complexity: In the previous analysis, we have broken down the time complexity for each step of the RSVD algorithm. To summarize, the overall time complexity of RSVD is approximately
$$\mathcal{T}_{\text{RSVD}} = \mathcal{O}(2dmp + 2dp^2 + mp^2). \tag{13}$$
In our implementation, we set the projection dimension p to twice the maximal rank r based on the RSVD algorithm proposed in [22], where p is significantly smaller than the matrix dimensions.

In comparison, the time complexity of the classical SVD for a matrix $\Delta W \in \mathbb{R}^{d\times m}$ is
$$\mathcal{T}_{\text{SVD}} = \mathcal{O}(\min(d^2\, m, dm^2)). \tag{14}$$
The computation cost of SVD can become prohibitively expensive for large matrices.

As illustrated in Fig. 4, we test the computation time of both RSVD and classical SVD for matrices with varying $N \times N$ dimensions where $N \in \{256, 512, 1024, 2048, 4096, 8192, 16384\}$. The results clearly show that as the matrix size increases, the computation time for classical SVD grows rapidly, reaching several minutes for larger matrices. For instance, when the matrix dimensions are $d = 16384$ and $m = 16384$, SVD takes over 173.53 s, while RSVD takes only about 0.038 s. Besides, we can see that the computation time of RSVD does not increase significantly as the matrix dimensions grow. This is primarily due to the choice of the projection dimension p, which is typically much smaller than the matrix

Algorithm 1 ECLoRA

Require: Total number of clients N; involved LoRA ranks set $\mathcal{R}$; local LoRA rank $r_i \in \mathcal{R}$ and local dataset $\mathcal{D}_i$ for the ith client; participation rate γ; compensation factor β; number of communication rounds T.
Initialization: Initialize the global LoRA parameters $\theta^{(0)}$ with the maximal rank in $\mathcal{R}$.
1 **for** $t = 0, \ldots, T-1$ communication rounds **do**
2 Sample a subset $\mathcal{S}^t$ of clients with $\left|\mathcal{S}^t\right| = \lceil \gamma N \rceil$.
3 **for** each client $i \in \mathcal{S}^t$ **in parallel do**
4 Broadcast the global LoRA $\theta^{(t-1)}[: r_i]$ corresponding to the client's LoRA rank r_i.
5 $\theta_i^{(t)} \leftarrow$ fine-tune $\theta^{(t-1)}[: r_i]$ with local data.
6 **end for**
7 $\{\Delta W_i^{(t)}\}_{i\in\mathcal{S}^t} \leftarrow \text{Rec}\left(\{\theta_i^{(t)}\}_{i\in\mathcal{S}^t}\right)$ the server receives the client parameters and reconstructs their weight matrices by multiplying A and B.
8 $\Delta W^{(t)} \leftarrow \sum_{i=1}^{|\mathcal{S}^t|} \frac{|\mathcal{D}_i|}{\sum_{j=1}^{|\mathcal{S}^t|}|\mathcal{D}_j|}\Delta W_i^{(t)}$ aggregation.
9 $\Delta\hat{W}^{(t)} \leftarrow \Delta W^{(t)} + \beta e^{(t)}$ error compensation.
10 $\theta^{(t)} \leftarrow$ decompose $\Delta\hat{W}^{(t)}$ by randomized SVD.
11 $e^{(t+1)} \leftarrow \Delta\hat{W}^{(t)} - \text{Rec}\left(\theta^{(t)}\right)$ update the error in the current round.
12 **end for**

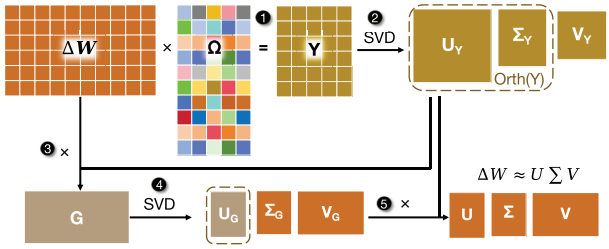

Fig. 3. Workflow of RSVD. (1) Multiply W with a random matrix Ω. (2) Perform SVD on Y to get the orthonormal basis. (3) Project W onto the orthogonal basis. (4) Apply SVD to the intermediate matrix G. (5) Combine results to form the final matrix.

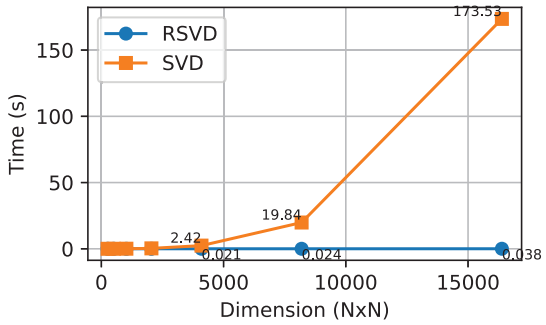

Fig. 4. Decomposition time of RSVD and SVD under different matrix dimensions.

dimensions d and m. The overall time complexity of RSVD is $\mathcal{O}(2dmp + 2dp^2 + mp^2)$, and while d and m may increase, the key computational steps in RSVD are all influenced by the projection dimension p. Since p is usually set as a small constant, the growth in computation time remains moderate compared to the classical SVD, where the complexity grows exponentially with the matrix size. This allows RSVD to maintain relatively acceptable computational costs, even as matrix dimensions grow, making it significantly more efficient than SVD for large-scale problems.

C. Theoretical Analysis

1) Generalization Ability: We analyze the generalization ability of RSVD in our method ECLoRA by referring to the discussion on SVD in [17], which extends Baxter's model of learning [35]. Here, $h_{\Delta W}(\cdot)$ represents the hypothesis generated by the model with LoRA weights ΔW, and $f(\Delta W; (x, y))$ denotes the loss function for a single data point (x, y). The expected loss is denoted as $L(\Delta W) \triangleq \mathbb{E}_{(x,y)\sim\mathcal{D}_i}[f(\Delta W; (x, y))]$. Our analysis is based on the following assumptions.

Assumption 1: The following Lipschitz conditions hold: $|f(\Delta W; x, y) - f(\Delta W'; x, y)| \leq L_f \|\Delta W - \Delta W'\|$ and $\|h(\Delta W; x) - h(\Delta W'; x)\| \leq L_h \|\Delta W - \Delta W'\|$.

Assumption 2: The LoRA weights can be bounded in a ball with radius R.

The first assumption is widely used in FL [36], [37], [38], which indicates that f and h are Lipschitz continuous with respect to the LoRA weights ΔW, ensuring the stability of the loss landscape. The second assumption is also commonly used in model generalization analysis [17], [39], [40], bounding the parameter space. Based on the two assumptions, we present the following theorem to reveal the generalization ability of ECLoRA.

Theorem 1: Let RSVD$(\Delta W, r)$ be the RSVD function using the top r singular values and the corresponding singular vectors to approximate ΔW. With probability at least $1 - \delta$, there exists a sample size $\tilde{N}$ such that for all ΔW, the bound $\|L(\Delta W) - L(\Delta W')\| \leq \epsilon$ holds when the number of local data samples for each client k exceeds $\tilde{N}$. $\tilde{N}$ is

$$\tilde{N} = \mathcal{O}\left(\frac{dm}{|\mathcal{C}|\epsilon^2} \log\left(\frac{RL_f L_h}{\epsilon - L_f L_h \phi}\right) - \frac{\log \delta}{|\mathcal{C}|\epsilon^2}\right) \tag{15}$$

where $\phi = (2+4(2\min\{d, m\}/(r-1))^{1/2})^{(1/(2q+1))}(\sigma_{r+1}+\sigma'_{r+1})$, q is the number of steps in a power iteration, and σ_{r+1} and σ'_{r+1} are the $(r+1)$th singular values of ΔW and $\Delta W'$.

Proof: Let $\mathbb{H}$ denote the function space with its elements parameterized by the LoRA weights $\Delta W \in \mathbb{R}^{d\times m}$, and the distance metric τ is defined as

$$\begin{aligned}
&\tau\left(\Delta W^r - \Delta W^{r'}\right) \\
&= \frac{1}{n}\mathbb{E}_{x,y\sim\mathcal{D}_i}\left[\left|\sum\left(f\left(\Delta W^r; x, y\right) - \sum f\left(\Delta W^{r'}; x, y\right)\right|\right] \\
&\overset{(a)}{\leq} L_f \|\Delta W^r - \Delta W^{r'}\| \\
&\overset{(b)}{\leq} L_f L_h \|\mathrm{RSVD}\left(\Delta W, r\right) - \mathrm{RSVD}\left(\Delta W', r\right)\| \\
&= L_f L_h \|\mathrm{RSVD}\left(\Delta W, r\right) - \Delta W - \mathrm{RSVD}\left(W', r\right) + \Delta W' \\
&\quad + \Delta W - \Delta W'\| \\
&\overset{(c)}{\leq} L_f L_h \|\mathrm{RSVD}\left(\Delta W, r\right) - \Delta W\| \\
&\quad + L_f L_h \|\mathrm{RSVD}\left(\Delta W', r\right) - \Delta W'\| \\
&\quad + L_f L_h \|\Delta W - \Delta W'\|
\end{aligned} \tag{16}$$

where (a) and (b) follow from Assumption 1 and (c) follows from the triangle inequality $\|a + b\| \leq \|a\| + \|b\|, \forall a, b \in \mathbb{R}^n$ [41], [42]. The above derivation is similar to the discussion in [17].

Then, for our further derivation, we quote the lemma about the error bound of RSVD in [22] as follows.

Lemma 1 [22]: Suppose that W is a real $d\times m$ matrix. Select an exponent q and a target number r of singular vectors, where $2 \leq r \leq 0.5\min\{d, m\}$. Execute the RSVD algorithm to obtain a rank-$2r$ factorization $U\Sigma V^*$. We replace the diagonal factor Σ by the matrix $\Sigma^{(r)}$ formed by zeroing out all but the largest k entries of Σ. For this truncated SVD, we have the error bound

$$\mathbb{E}\|W - U\Sigma_{(r)}V^*\| \leq \sigma_{r+1} + \left(1 + 4\sqrt{\frac{2\min\{d, m\}}{r-1}}\right)^{\frac{1}{2q+1}} \sigma_{r+1}$$

where $\mathbb{E}$ denotes expectation with respect to the random test matrix and σ_{r+1} is the $(r+1)$th singular value of W.

Based on the error bound of the truncated RSVD approximation in Lemma 1, we can continue to derive inequality (16)

$$\begin{aligned}
&\tau\left(\Delta W^r - \Delta W^{r'}\right) \\
&\leq L_f L_h\left(\sigma_{r+1} + \sigma'_{r+1}\right) \\
&\quad + L_f L_h\left(1 + 4\sqrt{\frac{2\min\{d, m\}}{r-1}}\right)^{\frac{1}{2q+1}}\left(\sigma_{r+1} + \sigma'_{r+1}\right) \\
&\quad + L_f L_h \|\Delta W - \Delta W'\|.
\end{aligned}$$

We can get an ϵ-covering in metric $\tau(\Delta W^r - \Delta W^{r'})$ if we select a covering in the parameter space with $\|\Delta W - \Delta W'\|$ equal to $(\epsilon/L_f L_h) - (2 + 4(2\min\{d, m\}/(r-1))^{1/2})^{(1/(2q+1))}(\sigma_{r+1} + \sigma'_{r+1})$. For clarity, let $\phi = (2 + 4(2\min\{d, m\}/(r-1))^{1/2})^{(1/(2q+1))}(\sigma_{r+1} + \sigma'_{r+1})$. Then, we get the covering number of $\mathbb{H}^{|\mathcal{C}|}$, denoted as $\log \mathcal{B}(\epsilon, \mathbb{H}^{|\mathcal{C}|})$, which is

$$\log \mathcal{B}(\epsilon, \mathbb{H}^{|\mathcal{C}|}) = \mathcal{O}\left(dm \log\left(\frac{RL_f L_h}{\epsilon - L_f L_h \phi}\right)\right)$$

where d and m are the dimensions of LoRA weights ΔW, $|\mathcal{C}|$ is the total number of clients, and R is the radius is the radius that bounds the LoRA weights within a ball as in Assumption 2.

According to previous studies [17], [35], [39], [40], there exists $\tilde{N}$ such that $\tau(\Delta W - \Delta W') \leq \epsilon$ holds for all ΔW, and $\tilde{N}$ is

$$\tilde{N} = \mathcal{O}\left(\frac{1}{|\mathcal{C}|\epsilon^2}\log\frac{\mathcal{B}\left(\epsilon, \mathbb{H}^{|\mathcal{C}|}\right)}{\delta}\right) = \mathcal{O}\left(\frac{dm}{|\mathcal{C}|\epsilon^2}\log\left(\frac{RL_f L_h}{\epsilon - L_f L_h \phi}\right) - \frac{\log\delta}{|\mathcal{C}|\epsilon^2}\right).$$

This completes the proof of Theorem 1. □

Theorem 1 presents the generalization bound of the RSVD in ECLoRA, which depends on the LoRA rank r. Specifically, as the rank r increases, ϕ is smaller, resulting in a smaller $\tilde{N}$ with better generalization. In our implementation, r specifically refers to the maximum of the involved ranks.

2) Convergence With EC: In ECLoRA, we perform RSVD on the reconstructed update matrix at the server, which serves as a form of compression. In the previous work, Li and Li [43] have provided the convergence rate $\mathcal{O}(1/\sqrt{TKN})$ of FL with error-compensated compression at the client side, where K is the number of local steps. All compression methods conducted at the client side with EC are applicable to their analysis. However, it should be noted that ECLoRA introduces compression via RSVD at the server side. Therefore, we basically follow the analysis in [43] to derive the convergence rate of FL with EC conducted at the server side, which is applicable to ECLoRA.

Assumption 3: The loss function f is L-smooth: $\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|$.

Assumption 4: For $\forall t \in [T], \forall i \in [N], \forall k \in [K]$: 1) the stochastic gradient is unbiased: $\mathbb{E}[g_{t,i}^{(k)}] = \nabla f_i(W_{t,i}^{(k)})$; 2) the local variance is bounded: $\mathbb{E}[\|g_{t,i}^{(k)} - \nabla f_i(W_{t,i}^{(k)})\|^2] \leq \xi^2$; and 3) the global variance is bounded: $(1/N)\sum_{i=1}^{N}\|\nabla f_i(W_t) - \nabla f(W_t)\|^2 \leq \xi_g^2$.

Assumption 5: It holds that $\|g_{i,k}^{(t)}\| \leq G, \forall t > 0, \forall i \in [n], \forall k \in [K]$.

Definition 1: The biased $q_{\mathcal{Q}}$-deviate compressor $\mathcal{Q}\colon \mathbb{R}^d \to \mathbb{R}^d$ is defined such that for $\forall x \in \mathbb{R}^d$, $\exists 0 \leq q_{\mathcal{Q}} < 1$ such that

$$\|\mathcal{Q}(x) - x\|_2^2 \leq q_{\mathcal{Q}}^2\|x\|_2^2.$$

Similarly as in [43], we newly add the above common assumptions and definitions to support our following convergence analysis. We clarify some new notations. Let ΔW be the reconstructed global update, and let $\Delta\tilde{W} = \mathrm{Rec}(\theta)$ be the compressed global update after RSVD. Consider the virtual sequence

$$\begin{aligned} x_{t+1} &= W_{t+1} - e_{t+1} = W_t - \Delta\tilde{W}_t - e_{t+1} \\ &= W_t - \Delta\tilde{W}_t - (\Delta W_t + e_t - \Delta\tilde{W}_t) = x_t - \Delta W_t. \end{aligned} \tag{17}$$

The pattern of (17) has been aligned with the proof of the theorem in [43]. Therefore, based on the derivation of theorem [43, Th.2], we replace η and q in [43] with 1 and $q_{\mathcal{Q}}$ in our case. We can directly derive the bound of the gradient norm from the following theorem.

Theorem 2: Let $W^* = \arg\min f(W)$ and $C_1 := 2 + (4q_{\mathcal{Q}}^2/((1 - q_{\mathcal{Q}}^2)^2))$. Under the above assumptions and definitions, when $\eta_l \leq (1/(2KL \cdot \max\{4, (C_1 + 1)\}))$, the squared gradient norm of FL with EC at the server side can be bounded by

$$\frac{1}{T}\sum_{t=1}^{T}\mathbb{E}\left[\|\nabla f(W_t)\|^2\right] \leq \frac{f(W_1) - f(W^*)}{\eta_l TK} + \frac{2\eta_l C_1 L}{N}\xi^2 + 10\eta_l^3 C_1 K^2 L^3\left(\xi^2 + 6K\xi_g^2\right) \tag{18}$$

where K is the local training steps. If we choose $\eta_l = \Theta(1/K\sqrt{T})$, then

$$\frac{1}{T}\sum_{t=1}^{T}\mathbb{E}\left[\|\nabla f(W_t)\|^2\right] = \mathcal{O}\left(\frac{f(\theta_1) - f(\theta^*)}{\sqrt{TKN}} + \frac{1}{\sqrt{TKN}}\xi^2 + \frac{\sqrt{N}}{T^{3/2}\sqrt{K}}\left(\xi^2 + K\xi_g^2\right)\right).$$

Theorem 2 is a variant of the theorem in [43], which shows that the convergence rate of FL with EC at the server side is also $\mathcal{O}(1/\sqrt{TKN})$. In comparison, compression without EC introduces a constant term as analyzed in [43], slowing the convergence rate. Theorem 2 elaborates the advantage of applying EC in the case of compression at the server side, in which ECLoRA lies.

3) Range of Compensation Factor β: Here, we give theoretical guidance for selecting the compensation factor β. The decomposition error in the $t+1$ round is

$$\begin{aligned} \|e^{(t+1)}\| &= \|\Delta\hat{W}^{(t)} - \mathrm{Rec}(\theta^{(t)})\| \\ &= \|(\Delta W^{(t)} + \beta e^{(t)}) - \mathcal{Q}(\Delta W^{(t)} + \beta e^{(t)})\| \\ &\overset{(a)}{\leq} q_{\mathcal{Q}}\|\Delta W^{(t)} + \beta e^{(t)}\| \\ &\overset{(b)}{\leq} q_{\mathcal{Q}}\left(\|\Delta W^{(t)}\| + \|\beta e^{(t)}\|\right) \\ &\leq q_{\mathcal{Q}}\|\Delta W^{(t)}\| + q_{\mathcal{Q}}\beta\|e^{(t)}\| \end{aligned} \tag{19}$$

where (a) follows from Definition 1 and (b) follows from the triangular inequality $\|a + b\| \leq \|a\| + \|b\|$ [41], [42].

From Assumption 5, we can bound the weight update

$$\|\Delta W^{(t)}\| = \left\|\sum_{k=0}^{k=K}\eta_l g_k^{(t)}\right\| \leq \sum_{k=0}^{k=K}\eta_l\left\|g_k^{(t)}\right\| \leq \eta_l KG \leq G_u \tag{20}$$

where η_l is the bounded learning rate defined in Theorem 2 and G_u is a constant bounding the weight update.

Substituting inequality (20) into inequality (19), we have

$$\|e^{(t+1)}\| \leq q_{\mathcal{Q}}G_u + q_{\mathcal{Q}}\beta\|e^{(t)}\|. \tag{21}$$

Considering $\|e^{(t)}\|$ and expanding the recursion, we have

$$\begin{aligned} \|e^{(t)}\| &\leq q_{\mathcal{Q}}G_u + q_{\mathcal{Q}}\beta\|e^{(t-1)}\| \\ &\leq q_{\mathcal{Q}}G_u + q_{\mathcal{Q}}\beta\left((q_{\mathcal{Q}}\eta_{t-1}KG + q_{\mathcal{Q}}\beta\|e^{(t-2)}\|)\right) \\ &\leq q_{\mathcal{Q}}G_u\left(1 + q_{\mathcal{Q}}\beta + (q_{\mathcal{Q}}\beta)^2 + \cdots + (q_{\mathcal{Q}}\beta)^{t-1}\right) \\ &\quad + (q_{\mathcal{Q}}\beta)^t\|e^{(0)}\|. \end{aligned} \tag{22}$$

To bound the decomposition error $\|e^{(t)}\|$, we need the geometric series $\sum_{i=0}^{\infty}(q_{\mathcal{Q}}\beta)^i$ to converge

$$\sum_{i=0}^{\infty}(q_{\mathcal{Q}}\beta)^i = \frac{1}{1 - q_{\mathcal{Q}}\beta} \tag{23}$$

which converges if and only if $|q_{\mathcal{Q}}\beta| < 1$.

For error stability and boundedness, we require $q_{\mathcal{Q}}\beta < 1$, i.e., $\beta < (1/q_{\mathcal{Q}})$. If this condition is met, the steady-state error is bounded by

$$\|e^{(\infty)}\| \leq \frac{q_{\mathcal{Q}} G_u}{1 - q_{\mathcal{Q}}\beta}. \tag{24}$$

Thus, the compensation factor β must satisfy the following range:

$$0 < \beta < \frac{1}{q_{\mathcal{Q}}}. \tag{25}$$

From Definition 1, we know that $0 \leq q_{\mathcal{Q}} < 1$. The value of $q_{\mathcal{Q}}$ is determined by the compression technique and the sparsity of the data. It is advisable to choose $\beta \in (0, 1]$ since inequality (25) will hold regardless of the value of $q_{\mathcal{Q}}$. In addition to its effect on the bound of the decomposition error, β controls the extent to which the decomposition error from the previous round is compensated in the current round. A small value of β leads to insufficient compensation for the lost information, while a large β may result in overcompensation, potentially causing instability in the training process. Therefore, $\beta = 1$ is often a reasonable and effective choice, as it compensates for the exact amount of information lost in the previous round [44]. In Section IV, we will explore learning curves under different values of β.

In summary, we present the generalization bound of RSVD in ECLoRA through Theorem 1, illustrating that a larger maximum of the involved ranks r in ECLoRA leads to a better generalization. Besides, we present the convergence rate $\mathcal{O}(1/\sqrt{TKN})$ of FL with EC conducted at the server side through Theorem 2, which is the case of ECLoRA. While ECLoRA employs LoRA for fine-tuning, which differs from the traditional full-rank SGD optimization in Theorem 2, our results provide valuable insights into the convergence trends of ECLoRA. As theoretical analysis specific to LoRA tuning is currently underexplored, more rigorous analysis along this line is left to our future work. Furthermore, the theoretical range of the compensation factor β ensures that the decomposition error remains bounded and stable during training, providing practical guidance for selecting β in real-world implementations of ECLoRA.

IV. Experiments

A. Setup

We conduct experiments on various natural language processing tasks, including natural language inference (RTE [26] and QNLI [26]), sentence pair semantic similarity (MRPC [26]), grammatical acceptability judgment (CoLA [26]), question answering (SQuAD [27]), and named entity recognition (CoNLL-2003 [28]). These tasks cover a wide range of language understanding challenges, such as reasoning, classification, reading comprehension, and sequence labeling. We take the following methods as our baselines.

1) *FedIT [16]:* FedIT aggregates the homogeneous LoRA module weight of each client by averaging, which limits the local LoRA rank to meet the lowest resource constraint.
2) *HETLORA [20]:* HETLORA employs zero-padding on all the LoRA matrices based on the largest rank, then conducts elementwise averages like FedAvg, and, finally, truncates the aggregated model to fit the local client LoRA rank.
3) *FLoRA [18]:* FLoRA stacks the heterogeneous LoRA modules from the participants on the server side and broadcasts the stacked LoRA to all clients in each round.
4) *FlexLoRA [17]:* FlexLoRA reconstructs the full-rank LoRA update for aggregation and employs SVD for weight redistribution.

We evaluate our proposed method ECLoRA and all other methods on $\text{RoBERTa}_{\text{BASE}}$ [23] with 125M pretrained parameters, $\text{RoBERTa}_{\text{LARGE}}$ [23] with 355M pretrained parameters, $\text{DeBERTa}_{\text{XXL}}$ [24] with 1.5B parameters, and $\text{LLaMA-3.1}_{\text{8B}}$ [25]. The backbones of these models are frozen for all methods. The target modules of LoRA are set to all the linear layers in the attention blocks by default.

All FL experiments are conducted on an Ubuntu 22.04 machine with four NVIDIA GeForce RTX 4090 GPUs, one 13th-Gen Intel[2] Core[3] i9-13900K CPU and 125-GB memory. In our experiments, we set a total of 100 clients, with a participation rate of 0.1 in each round. For FedIT, each client is assigned a homogeneous LoRA rank of 4. For the other baselines and our method, each client is initially assigned a heterogeneous LoRA rank, randomly selected from the set {4, 8, 16, 32, 64}. The total number of communication rounds is set to 200 by default. We adopt the AdamW [45] optimizer for local training. The learning rate is set to $2e^{-4}$, accompanied with a linear scheduler, which warm-ups it to the initial learning rate during the first three rounds and decays it to 0 over rounds. The batch size is set at 8, and the maximum token length is 384 for SQuAD and 128 for the other tasks. The compensation factor β is set to 1. All the experiments are repeated with three random seeds, and we report the standard deviations.

B. Overall Evaluation

1) Model Accuracy: Table II shows the performance of all the methods on the various models across multiple benchmark tasks. It can be seen that ECLoRA consistently achieves the highest performance across a variety of datasets, demonstrating its robustness. For example, in the MRPC dataset, ECLoRA outperforms all other methods in each model category, achieving the top score of 88.7 with $\text{RoBERTa}_{\text{base}}$, 89.7 with $\text{RoBERTa}_{\text{large}}$, and 88.9 with $\text{DeBERTa}_{\text{XXL}}$. Similarly, in the RTE dataset, ECLoRA significantly outperforms other methods, particularly with $\text{RoBERTa}_{\text{large}}$ and $\text{DeBERTa}_{\text{XXL}}$, scoring 82.3 and 87.7, respectively. Across most tasks, ECLoRA offers superior accuracy compared with the other methods. Notably, although FLoRA directly stacks and transmits the exact local LoRA modules to each client, it performs comparably or even worse than FedIT. This phenomenon has also been observed in prior work [34], which attributes it to the need for reinitializing LoRA modules in every round.

[2]Registered trademark.
[3]Trademarked.

TABLE II

PERFORMANCE COMPARISON OF FIVE METHODS ACROSS VARIOUS MODELS AND TASKS. WE REPORT THE MATTHEW'S CORRELATION FOR COLA, $F1$ SCORE FOR CONLL03, EXACT MATCH FOR SQUAD, AND ACCURACY FOR OTHER TASKS, PRESENTING THE BEST EVALUATION METRIC FOR EACH

Model	Method	MRPC	CoLA	QNLI	RTE	SQuAD	CoNLL03	Avg.	Agg. Time
$\text{RoBERTa}_{\text{base}}$	FedIT	83.8±1.1	52.8±0.8	91.0±0.5	63.5±1.2	84.2±0.7	94.0±1.1	78.2	0.02s
	HETLORA	85.1±1.3	54.0±1.1	91.0±0.3	64.6±1.5	84.2±0.8	94.5±1.0	78.9	0.03s
	FLoRA	83.0±1.2	55.2±1.3	90.9±0.5	63.8±1.4	82.8±0.6	93.8±1.0	78.3	0.03s
	FlexLoRA	87.5±1.5	**55.8**±1.6	**91.2**±0.6	72.9±1.3	**84.4**±0.7	**94.9**±1.2	81.1	9.64s
	ECLoRA	**88.7**±1.4	55.3±1.0	**91.2**±0.5	**73.7**±1.2	**84.4**±0.6	**94.9**±1.3	**81.4**	0.33s
$\text{RoBERTa}_{\text{large}}$	FedIT	86.2±0.9	61.1±1.1	93.7±0.5	68.6±1.6	88.3±0.6	95.6±0.8	82.2	0.04s
	HETLORA	86.6±1.0	59.8±1.3	93.8±0.4	67.5±1.4	**88.4**±0.6	95.8±1.0	81.9	0.06s
	FLoRA	87.2±1.1	61.8±1.2	93.5±0.4	71.8±1.5	87.4±0.5	94.8±1.1	82.7	0.05s
	FlexLoRA	88.7±1.2	62.4±1.5	93.8±0.6	78.3±1.7	**88.4**±0.6	**96.0**±1.1	84.6	15.25s
	ECLoRA	**89.7**±1.1	**62.9**±1.1	**93.9**±0.6	**82.3**±1.4	**88.4**±0.6	95.8±0.7	**85.5**	0.52s
$\text{DeBERTa}_{\text{XXL}}$	FedIT	87.5±1.1	**70.9**±1.5	95.6±0.2	83.3±1.5	90.6±0.4	96.4±1.1	87.4	0.09s
	HETLORA	85.2±1.5	69.0±1.4	95.5±0.3	84.1±1.5	90.7±0.3	96.4±1.0	86.8	0.13s
	FLoRA	84.8±1.3	68.0±1.4	95.4±0.2	72.9±1.6	90.6±0.3	95.6±0.9	84.5	0.11s
	FlexLoRA	87.9±1.3	70.0±1.6	95.5±0.2	87.4±1.7	90.7±0.3	96.3±1.2	87.9	71.51s
	ECLoRA	**88.9**±1.0	**70.9**±1.0	**95.7**±0.3	**87.7**±1.3	**90.8**±0.3	**96.6**±1.0	**88.4**	1.65s
$\text{LLaMA-3.1}_{\text{8B}}$	FedIT	85.0±1.5	66.1±1.5	94.9±0.6	84.8±1.0	81.3±0.4	83.2±1.2	82.5	0.10s
	HETLORA	86.0±1.2	67.0±1.4	**95.3**±0.5	86.6±1.3	81.6±0.5	83.4±1.1	83.3	0.18s
	FLoRA	84.8±1.4	65.3±1.5	95.1±0.5	81.2±1.6	80.4±0.4	83.0±1.2	81.6	0.13s
	FlexLoRA	**89.2**±1.4	67.2±1.6	95.2±0.5	87.0±1.4	**81.9**±0.6	83.7±1.0	84.0	261.83s
	ECLoRA	88.2±1.2	**68.3**±1.4	**95.3**±0.4	**88.1**±1.5	**81.9**±0.6	**83.9**±1.1	**84.2**	7.87s

TABLE III

SPEEDUP GAINS ON THE CERTAIN TASKS AND THE AVERAGE SPEEDUP GAINS ON ALL THE TASKS OF HETLORA, FLORA, FLEXLORA, AND ECLORA OVER FEDIT IN ROUND-TO-TARGET-ACCURACY UNDER THE FOUR MODELS

Model	Task	Target Acc.	Speedup			
			HETLORA	FLoRA	FlexLoRA	ECLoRA
$\text{RoBERTa}_{\text{base}}$	MRPC	83%	1.04×	1.10×	**3.73**×	**3.73**×
	CoLA	52%	**1.33**×	**1.33**×	**1.33**×	**1.33**×
	QNLI	90%	1.04×	0.79×	**1.36**×	**1.36**×
	RTE	63%	1.31×	1.77×	**7.78**×	**7.78**×
	SQuAD	82%	1.04×	0.68×	**1.97**×	**1.97**×
	CoNLL03	93%	1.11×	0.78×	1.65×	**1.83**×
	Avg.	-	1.15×	1.07×	2.98×	**3.01**×
$\text{RoBERTa}_{\text{large}}$	MRPC	85%	1.50×	1.56×	4.03×	**4.16**×
	CoLA	59%	1.25×	1.25×	**1.66**×	1.25×
	QNLI	93%	1.46×	1.47×	**1.69**×	1.46×
	RTE	67%	0.91×	2.07×	3.12×	**5.30**×
	SQuAD	87%	1.06×	0.68×	**1.30**×	**1.30**×
	CoNLL03	94%	1.19×	0.72×	**1.54**×	1.48×
	Avg.	-	1.23×	1.29×	2.22×	**2.49**×
$\text{DeBERTa}_{\text{XXL}}$	MRPC	84%	0.74×	0.86×	**2.61**×	2.32×
	CoLA	68%	0.60×	0.12×	**1.04**×	**1.04**×
	QNLI	95%	0.86×	0.39×	0.86×	**1.08**×
	RTE	72%	1.00×	0.44×	2.19×	**2.63**×
	SQuAD	90%	**1.07**×	0.82×	**1.07**×	**1.07**×
	CoNLL03	95%	0.91×	0.51×	**1.40**×	**1.40**×
	Avg.	-	0.86×	0.52×	1.53×	**1.59**×
$\text{LLaMA-3.1}_{\text{8B}}$	MRPC	84%	1.26×	0.85×	**2.77**×	2.32×
	CoLA	64%	1.00×	0.64×	**1.60**×	**1.07**×
	QNLI	94%	1.00×	0.67×	**1.20**×	**1.20**×
	RTE	81%	1.00×	0.56×	**1.64**×	**1.64**×
	SQuAD	80%	1.00×	0.56×	**1.04**×	**1.04**×
	CoNLL03	83%	1.50×	0.51×	1.85×	**1.95**×
	Avg.	-	1.13×	0.63×	**1.68**×	1.54×

ECLoRA also exhibits significant advantages across different model sizes. For example, with the $\text{RoBERTa}_{\text{large}}$ model, ECLoRA achieves the highest performance in tasks such as RTE (82.3) and CoLA (62.9), outperforming all other methods. Its dominance is even more pronounced in the $\text{DeBERTa}_{\text{XXL}}$ model, where it scores the highest in nearly all tasks, including QNLI (95.7) and CoLA (70.9). Compared to methods such as FedIT and HETLORA, which generally perform well but fall short in certain tasks, ECLoRA provides consistently better results, proving its ability to leverage both

small and large models effectively for improved task-specific performance.

2) Model Convergence: As for model convergence, we compare the speedup gains of HETLORA, FLoRA, FlexLoRA, and ECLoRA over FedIT in round-to-target-accuracy. Table III shows the speedup gains of these methods on the six tasks on RoBERTa$_{\text{base}}$, RoBERTa$_{\text{large}}$, DeBERTa$_{\text{XXL}}$, and LLaMA-3.1$_{\text{8B}}$. For the RoBERTa$_{\text{base}}$ model, ECLoRA achieves an average speedup of 3.01×, which is slightly higher than FlexLoRA's 2.98×, while HETLORA and FLoRA lag significantly behind at 1.15× and 1.07×, respectively. Particularly, on tasks such as RTE and MRPC, both FlexLoRA and ECLoRA achieve remarkable speedups of 7.78× and 3.73×, respectively, highlighting their efficiency in significantly reducing the number of rounds required to reach the target accuracy. When evaluating the larger RoBERTa$_{\text{large}}$ model, a similar trend is observed, with ECLoRA leading the performance gains. ECLoRA achieves an average speedup of 2.49×.

On larger models such as DeBERTa$_{\text{XXL}}$ and LLaMA-3.1$_{\text{8B}}$, ECLoRA continues to outperform the other methods in most cases. For the DeBERTa$_{\text{XXL}}$ model, ECLoRA achieves an average speedup of 1.59×, slightly higher than FlexLoRA's 1.53× and significantly better than HETLORA's 0.86× and FLoRA's 0.52×, which even shows a slowdown in some tasks such as MRPC. On the LLaMA-3.1$_{\text{8B}}$ model, ECLoRA's average speedup is 1.54×, with FlexLoRA leading at 1.68×, but still significantly faster than HETLORA's 1.13× and FLoRA's 0.63×. It can be observed that on the two larger models, FLoRA converges significantly slower across all datasets, indicating that the adverse effect of LoRA reinitialization becomes more pronounced with larger models. These results demonstrate that ECLoRA not only maintains strong performance on smaller models but also scales effectively to much larger models, consistently improving speedup gains. While FlexLoRA shows comparable convergence performance to ECLoRA, its extremely longer aggregation time limits practicality. We will analyze this issue in detail in the next paragraph.

3) Aggregation Time: The core distinction among HETLORA, FLoRA, FlexLoRA, and ECLoRA lies in their respective aggregation algorithms. Therefore, we report the aggregation time of these methods in Table II. While FlexLoRA exhibits similar convergence speeds to ECLoRA, it can be seen that the aggregation time of ECLoRA is much lower than that of FlexLoRA. For instance, in RoBERTa$_{\text{base}}$, ECLoRA costs an aggregation time of just 0.33 s, which is much faster than FlexLoRA's 9.64 s. Similarly, on larger models such as LLaMA-3.1$_{\text{8B}}$, ECLoRA outpaces FlexLoRA in aggregation time (7.87 versus 261.83 s) while maintaining highly competitive accuracy and convergence speed.

In summary, ECLoRA demonstrates outstanding performance in both accuracy and efficiency. Despite its rapid convergence, it retains a highly efficient aggregation process, achieving significantly shorter aggregation times compared to FlexLoRA. These advantages make ECLoRA a robust and practical solution for large-scale FL.

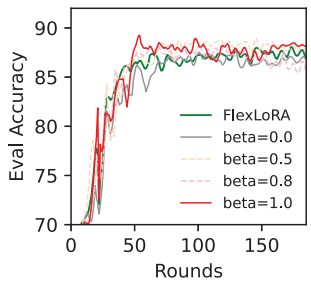

Fig. 5. Learning curves of FlexLoRA and ECLoRA with the different compensation factor β on RoBERTa$_{\text{large}}$ MRPC.

TABLE IV

PERFORMANCE OF THE FIVE METHODS IN HETEROGENEOUS LoRA-BASED FL WITH DIFFERENT RANK SETTINGS ON RoBERTa$_{\text{LARGE}}$ MRPC. THE FIRST TWO COLUMNS CORRESPOND TO SETTINGS WHERE THE CLIENT RANKS ARE SAMPLED FROM DISCRETE SETS. THE LAST TWO COLUMNS CORRESPOND TO SETTINGS WHERE THE RANKS ARE SAMPLED AS INTEGERS FROM CONTINUOUS INTERVALS

Method	{1,2,4,8,16,32}	{4,8,16,32,64}	[1, 32]	[4, 64]
FedIT	85.0	86.2	84.5	88.4
HETLORA	85.2	86.6	85.0	87.2
FLoRA	84.5	87.2	84.3	88.9
FlexLoRA	88.4	88.7	88.4	89.2
ECLoRA	**89.2**	**89.7**	**88.7**	**89.7**

C. In-Depth Evaluation

This section evaluates ECLoRA thoroughly with more experimental results on RoBERTa$_{\text{large}}$ MRPC to evaluate its robustness in diverse FL settings.

1) Impact of Compensation Factor: In this experiment, we explore the impact of the compensation factor $\beta \in \{0.0, 0.5, 0.8, 1.0\}$. The size of β represents the proportion of EC terms, where $\beta = 0.0$ means that we only apply RSVD but do not compensate for the error. This experiment helps to visualize the effect of the compensation mechanism. As shown in Fig. 5, compared to FlexLoRA with classical SVD, RSVD does reduce the convergence speed and accuracy according to the curve with $\beta = 0.0$. However, applying our compensation mechanism significantly improves performance, with the setting $\beta = 1.0$ achieving the best results. These observations are consistent with our theoretical discussion in Section III-C.

2) Impact of Heterogeneous Ranks: In this experiment, we explore the impact of the different heterogeneous rank settings on RoBERTa$_{\text{large}}$ MRPC. As shown in Table IV, the first and second columns represent the settings where each client's LoRA rank is randomly selected from a discrete set of values, specifically $\{1, 2, 4, 8, 16, 32\}$ and $\{4, 8, 16, 32, 64\}$, respectively. The third and fourth columns correspond to the settings where each client's LoRA rank is randomly sampled as an integer from the continuous intervals $[1, 32]$ and $[4, 64]$, respectively. We can see that when the maximum of the involved ranks increases from 32 to 64, the accuracy of all the methods is improved, which is aligned with Theorem 1. Among these methods, ECLoRA consistently achieves the

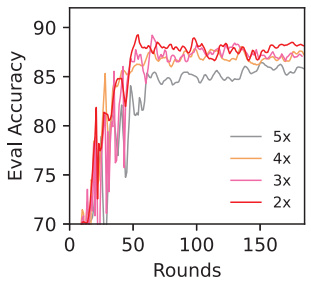

Fig. 6. Learning curves of ECLoRA on RoBERTa$_{\text{large}}$ MRPC with different projection dimensions p, expressed as multiples of the maximum LoRA rank among all clients.

TABLE V
PERFORMANCE OF THE FIVE METHODS WITH DIFFERENT PARTICIPATION RATES. THE PARTICIPATION RATE REPRESENTS THE PERCENTAGE OF CLIENTS PARTICIPATING IN EACH ROUND

Method	$\gamma = 0.1$	$\gamma = 0.3$	$\gamma = 0.6$	$\gamma = 1.0$
FedIT	86.2	78.0	86.7	87.5
HETLORA	86.6	80.4	87.0	87.2
FLoRA	87.2	77.2	85.0	88.2
FlexLoRA	88.7	86.2	88.9	**88.4**
ECLoRA	**89.7**	**87.9**	**89.4**	**88.4**

highest performance across various rank settings, such as 89.7% in the setting $\{4, 8, 16, 32, 64\}$. This adaptability is particularly beneficial in heterogeneous environments where different clients may have varying computational capacities, making ECLoRA a robust choice for practical applications.

3) Impact of the Projecting Dimension p: In this experiment, we explore the impact of the projecting dimension p used in ECLoRA's RSVD. In the default setting, the projecting dimension p is set to twice ($2\times$) the maximum LoRA rank across all clients. We further evaluate the performance when p is increased to $3\times$, $4\times$, and $5\times$ the maximum rank. As the projecting dimension p increases, the computation time for RSVD rises accordingly. This experiment aims to determine the optimal dimension for ECLoRA. As shown in Fig. 6, projecting the original weight matrix to twice the maximum rank achieves the fastest convergence and highest accuracy, indicating that a $2\times$ projection is most suitable.

4) Impact of Non-IID Degree: In this experiment, we explore the impact of different non-IID degrees with the Dirichlet alpha α in {0.6, 1, 10, 100} on RoBERTa$_{\text{large}}$ MRPC. Besides, we also present the result in the IID setting. As shown in Fig. 7, ECLoRA consistently achieves faster convergence compared to other methods under both IID and non-IID scenarios. As α increases, the learning curve fluctuates significantly less, which is consistent with a more even distribution of client data. When α is set at 0.6, 1.0, and 100.0, ECLoRA obviously shows the highest accuracy. In comparison, FedIT, HETLORA, and FLoRA have a much slower convergence speed. While FlexLoRA has a similar convergence performance to ECLoRA, the huge aggregation time leads to the impracticability of FlexLoRA.

5) Impact of the Number of Clients: In this experiment, we investigate the impact of the number of clients on the performance of various methods. We apply different participation rates γ to control the number of clients participating in each round. As shown in Table V, ECLoRA outperforms all other methods, with its accuracy peaking at 89.7% when only 0.1 of clients is sampled and remaining strong across higher participation rates, achieving 87.9% at 0.3, 89.4% at 0.6, and 88.4% at full client participation. The results reveal that ECLoRA consistently achieves the highest accuracy across all participation rates, demonstrating its robustness and reliability regardless of client participation.

TABLE VI
ACCURACY, ROUND-TO-TARGET-ACCURACY SPEEDUP, AGGREGATION TIME, AND FLOPS UNDER DIFFERENT ABLATION SETTINGS ON ROBERTA$_{\text{LARGE}}$ MRPC. THE SHADED ROW REPRESENTS OUR METHOD, ECLORA

Ablation Settings			Acc(%)	Speedup (T=86%)	Agg. Time	FLOPs
SVD	RSVD	EC				
✓	✗	✗	88.7	2.70×	15.25s	1236.9G
✓	✗	✓	89.9	2.83×	15.33s	1236.7G
✗	✓	✗	88.2	2.77×	0.46s	54.7G
✗	✓	✓	89.7	3.31×	0.52s	54.9G
FedIT			86.2	1.00×	0.04s	0.036G

6) Impact of the LoRA Modules: In this experiment, we explore the impact of different LoRA modules, including {"all-linear"}, {"q", "k", "v", "o"}, {"q", "k", "v"}, and {"q"}. The results in Fig. 8 demonstrate that ECLoRA maintains high convergence speed and accuracy across all module configurations. These results further demonstrate that ECLoRA is inherently robust and can deliver strong performance regardless of the specific choice of LoRA modules.

7) Impact of Epochs: In this experiment, we explore the impact of the local epochs and the global epochs (i.e., communication rounds). We set the local epochs in {1, 2, 3} and set the communication rounds in {200, 300, 400}, respectively, on RoBERTa$_{\text{large}}$ MRPC. Fig. 9 presents the best validation accuracy achieved by five different methods. We observe that increasing the number of local epochs generally improves the performance of most methods. Across all settings, ECLoRA consistently achieves the highest accuracy. Similarly, increasing the number of communication rounds leads to better accuracy for most methods although the accuracy gap between FedIT and ECLoRA narrows as rounds increase, indicating that FedIT requires more communication rounds to achieve competitive results. In contrast, FLoRA exhibits an unstable trend. Its accuracy improves when increasing local epochs from 1 to 2 and communication rounds from 200 to 300. However, further increasing the local epochs to 3 or the communication rounds to 400 leads to a clear degradation in performance. These results suggest that frequent LoRA reinitialization in FLoRA disrupts convergence during prolonged training.

D. Ablation Study

The core of ECLoRA lies in two key designs: RSVD for efficient low-rank approximation and EC for accuracy recovery. To validate the effectiveness of these components, we conduct an ablation study on RoBERTa$_{\text{large}}$ MRPC by

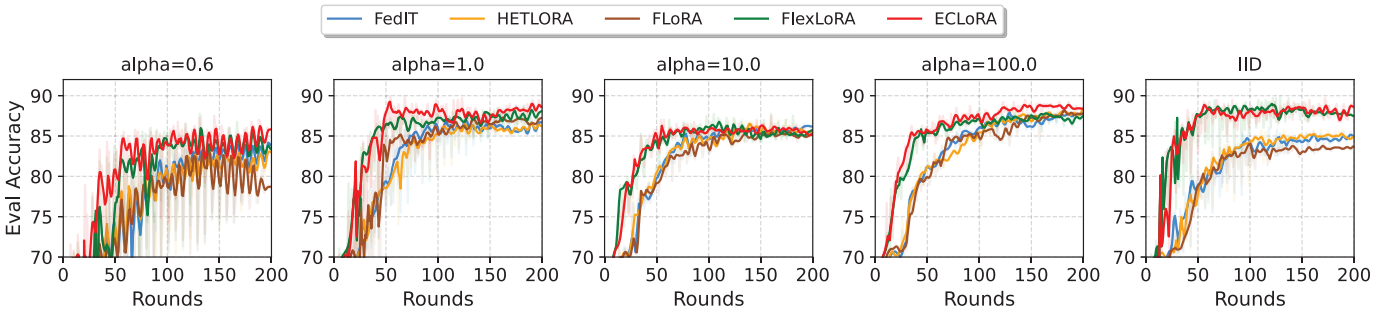

Fig. 7. Learning curves of the five methods on the IID setting and the non-IID settings with a different Dirichlet alpha on RoBERTa$_{\text{large}}$ MRPC. A larger alpha reflects that the data are more evenly distributed.

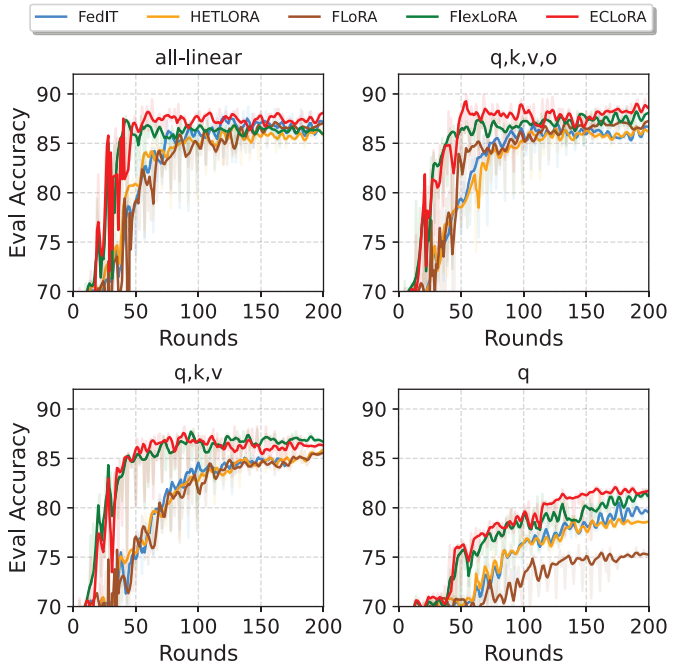

Fig. 8. Learning curves of the five methods on the settings with different LoRA target modules. "All-linear" refers to all the linear layers.

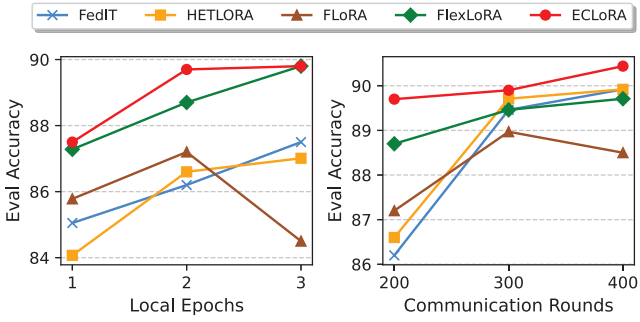

Fig. 9. Accuracy of the five methods under varying local epochs and communication rounds on RoBERTa$_{\text{large}}$ MRPC.

evaluating different configurations: using classical SVD or RSVD and with or without EC. Table VI summarizes the performance in terms of accuracy, round-to-target-accuracy speedup, aggregation time, and FLOPs.

On the one hand, replacing classical SVD with RSVD significantly improves efficiency. While classical SVD requires 15.25 s for aggregation and over 1200 GFLOPs, RSVD reduces the aggregation time to just 0.46 s and the computation to around 55 GFLOPs. The accuracy drop is minimal, from 88.7% to 88.2%, showing that RSVD offers substantial speedup with negligible performance loss. On the other hand, adding EC further boosts the accuracy. In both SVD and RSVD settings, EC consistently improves performance by 1.2% when using SVD and by 1.5% when using RSVD, while introducing almost no computational overhead. Notably, it also accelerates convergence, increasing the speedup from 2.77× to 3.31× when used with RSVD.

Overall, as shown in the shaded row of Table VI, ECLoRA, which combines RSVD and EC, achieves the best tradeoff between accuracy and efficiency, reaching 89.7% accuracy, a 3.31× speedup, and just 0.52 s of aggregation time. This demonstrates that ECLoRA offers superior performance while remaining computationally efficient.

TABLE VII

WEIGHTED AVERAGE ROUGE-L AND ROUGE-1 SCORES ON THE TEST SET OF UNSEEN CLIENTS IN THE SETTING OF FINE-TUNING THE *datajuicer/LLaMA–1B–dj–refine–150B* MODEL ON THE *Natural Instructions v2* DATASET. THE SCORES PROVIDE INSIGHTS INTO THE GLOBAL MODEL'S GENERALIZATION ABILITY

Metric	FedIT	HETLORA	FLoRA	FlexLoRA	ECLoRA
Rouge-L	57.0	57.0	57.1	57.6	**57.7**
Rouge-1	65.3	65.3	65.3	**65.5**	**65.5**

E. Case Studies

1) Multitask Local Data: In all the above experiments, the local data of each client are a subset of the same task dataset. In this experiment, we explore when the local data of each client belongs to different tasks and how the trained global model performs on unseen tasks. Specifically, as in [17], we select the LLaMA-1.3B from Data Juicer [46] as the foundation model and the AllenAI natural instruction dataset v2 as the dataset [47], which comprises over 1600 distinct natural language tasks. In our experiment, there are 1613 clients, and each client holds a unique task to mirror a task heterogeneous environment. The other hyperparameters are the same as those in our general experiments. Table VII shows the average Rouge-L and Rouge-1 scores on the test set of

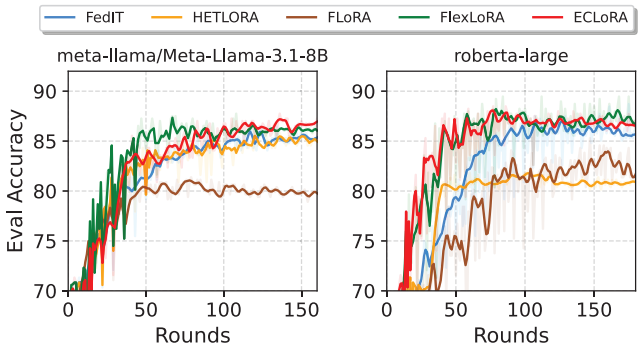

Fig. 10. Learning curves of five methods under dynamic client rank settings on MRPC using LLaMA-3.1_{8B} and $\text{RoBERTa}_{\text{large}}$.

TABLE VIII
ACCURACY UNDER DIFFERENT PROPORTIONS OF LABEL-FLIPPING ATTACKERS ON $\text{RoBERTa}_{\text{BASE}}$ MRPC

Metric	0	10%	15%	20%	25%
FedIT	83.8	80.1	79.9	80.3	79.1
HETLORA	85.1	79.4	80.3	80.3	79.9
FLoRA	83.0	73.0	72.7	70.8	71.5
FlexLoRA	87.5	82.3	80.8	82.8	79.1
ECLoRA	**88.7**	**85.2**	**83.8**	**83.0**	**83.8**

unseen clients, reflecting the global model's generalization ability. The results indicate that ECLoRA outperforms other methods in terms of both Rouge-L (57.7%) and Rouge-1 (65.5%) scores. This demonstrates its robust generalization capabilities in a heterogeneous task environment, effectively adapting to diverse unseen tasks while maintaining superior performance compared to FedIT, HETLORA, FLoRA, and FlexLoRA.

2) Dynamic Change in Ranks: In FL, the resource environment of clients (e.g., computational power, storage capacity, and network bandwidth) can fluctuate dynamically over time. In this experiment, we explore the case where each client's LoRA rank can dynamically change based on its current resource conditions in each round of FL. Specifically, in each round of FL, for any given client, we randomly select a rank from a predefined set of candidate ranks $\{4, 8, 16, 32, 64\}$ to serve as the client's LoRA rank for that round of local training. We fine-tune $\text{RoBERTa}_{\text{large}}$ and LLaMA-3.1_{8B} on MRPC and present the learning curves in Fig. 10. It can be seen that HETLORA struggles to adapt effectively to the fluctuating ranks, and its performance drops significantly on $\text{RoBERTa}_{\text{large}}$. On the other hand, ECLoRA maintains its advantage in both convergence speed and accuracy, demonstrating its robustness in handling dynamic rank adjustments.

3) Under Privacy Attacks: To further evaluate the robustness of ECLoRA against privacy threats, we simulate label-flipping attacks, a common type of data poisoning that can also degrade model privacy and integrity in FL. Specifically, we randomly select a proportion of malicious clients to flip the labels of their local data, thus injecting adversarial noise during training. Following standard practice, we vary the attacker proportion from 0% to 25%. Table VIII summarizes the accuracy under different attack intensities on $\text{RoBERTa}_{\text{base}}$ MRPC. ECLoRA consistently achieves the highest accuracy across all attack levels. Without any attackers, ECLoRA reaches 88.7% accuracy, outperforming all baselines. Even with 25% malicious clients, ECLoRA maintains 83.8% accuracy, showing strong resilience compared to methods such as FedIT with 79.1% and FlexLoRA with 79.1%. These results demonstrate that ECLoRA exhibits strong robustness under privacy-related attacks, highlighting its potential for practical deployment in FL systems.

V. CONCLUSION AND LIMITATION

In this article, we investigate applying heterogeneous LoRAs in federated fine-tuning, so as to address the "bucket effect" challenge in FL and improve model performance. Specifically, we propose a novel method, called ECLoRA, which enables efficient federated fine-tuning on heterogeneous LoRAs by leveraging RSVD for model aggregation and decomposition, along with an EC mechanism that corrects decomposition errors across rounds. Our theoretical analysis provides generalization and convergence support for ECLoRA and offers guidance for setting the compensation factor. Extensive experiments demonstrate that ECLoRA consistently outperforms state-of-the-art methods, achieving a $1.54\times$–$3.01\times$ convergence speedup and 1%–3%+ higher accuracy relative to FedIT, while maintaining minimal aggregation overhead. These results highlight the effectiveness and efficiency of ECLoRA in heterogeneous FL environments.

In terms of limitations, our method does not directly modify the RSVD algorithm itself, as we focus on leveraging RSVD to efficiently address the aggregation issue of heterogeneous LoRAs. While RSVD is a well-established technique, its adaptation to FL with heterogeneous LoRAs is the novel aspect of our approach. In future work, we will explore further improvements in the efficiency of model decomposition. In addition, further rigorous convergence analysis on EC for LoRA-based FL is a promising direction for future research.

REFERENCES

[1] E. Kasneci et al., "ChatGPT for good? On opportunities and challenges of large language models for education," *Learn. Individual Differences*, vol. 103, Mar. 2023, Art. no. 102274.
[2] A. J. Thirunavukarasu, D. S. J. Ting, K. Elangovan, L. Gutiérrez, T. F. Tan, and D. S. W. Ting, "Large language models in medicine," *Nature Med.*, vol. 29, no. 8, pp. 1930–1940, Jul. 2023.
[3] W. Xin Zhao et al., "A survey of large language models," 2023, *arXiv:2303.18223*.
[4] L. Wu et al., "A survey on large language models for recommendation," *World Wide Web*, vol. 27, no. 5, p. 60, Aug. 2024.
[5] D. Cheng, S. Huang, and F. Wei, "Adapting large language models via reading comprehension," in *Proc. 12th Int. Conf. Learn. Represent.*, Jan. 2023, pp. 1–9. [Online]. Available: https://openreview.net/forum?id=y886UXPEZ0
[6] N. Ding et al., "Parameter-efficient fine-tuning of large-scale pre-trained language models," *Nature Mach. Intell.*, vol. 5, no. 3, pp. 220–235, Mar. 2023.
[7] Z. Han, C. Gao, J. Liu, J. Zhang, and S. Qian Zhang, "Parameter-efficient fine-tuning for large models: A comprehensive survey," 2024, *arXiv:2403.14608*.
[8] R. He et al., "On the effectiveness of adapter-based tuning for pretrained language model adaptation," 2021, *arXiv:2106.03164*.
[9] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," 2021, *arXiv:2106.09685*.

[10] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2023, pp. 10088–10115.
[11] C. Zhang, Y. Xie, H. Bai, B. Yu, W. Li, and Y. Gao, "A survey on federated learning," *Knowl.-Based Syst.*, vol. 216, Jan. 2021, Art. no. 106775.
[12] P. Kairouz et al., "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, nos. 1–2, pp. 1–210, 2021.
[13] K. Wang et al., "FlexiFed: Personalized federated learning for edge clients with heterogeneous model architectures," in *Proc. ACM Web Conf.*, Apr. 2023, pp. 2979–2990.
[14] T. Guo, S. Guo, and J. Wang, "PFedPrompt: Learning personalized prompt for vision-language models in federated learning," in *Proc. ACM Web Conf.*, Apr. 2023, pp. 1364–1374.
[15] W. Ning et al., "One teacher is enough: A server-clueless federated learning with knowledge distillation," *IEEE Trans. Services Comput.*, vol. 17, no. 5, pp. 2704–2718, Sep. 2024.
[16] J. Zhang et al., "Towards building the federatedgpt: Federated instruction tuning," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Apr. 2024, pp. 6915–6919.
[17] J. Bai, D. Chen, B. Qian, L. Yao, and Y. Li, "Federated fine-tuning of large language models under heterogeneous tasks and client resources," 2024, *arXiv:2402.11505*.
[18] Z. Wang et al., "FLoRA: Federated fine-tuning large language models with heterogeneous low-rank adaptations," 2024, *arXiv:2409.05976*.
[19] T. Jiang et al., "MoRA: High-rank updating for parameter-efficient fine-tuning," 2024, *arXiv:2405.12130*.
[20] Y. J. Cho, L. Liu, Z. Xu, A. Fahrezi, M. Barnes, and G. Joshi, "Heterogeneous LoRa for federated fine-tuning of on-device foundation models," in *Proc. Int. Workshop Federated Learn. Age Found. Models Conjunct. NeurIPS*, 2023, pp. 1–6.
[21] V. Klema and A. Laub, "The singular value decomposition: Its computation and some applications," *IEEE Trans. Autom. Control*, vol. AC-25, no. 2, pp. 164–176, Apr. 1980.
[22] N. Halko, P. G. Martinsson, and J. A. Tropp, "Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions," *SIAM Rev.*, vol. 53, no. 2, pp. 217–288, Jan. 2011.
[23] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.
[24] P. He, X. Liu, J. Gao, and W. Chen, "DeBERTa: Decoding-enhanced BERT with disentangled attention," in *Proc. Int. Conf. Learn. Represent.*, Jan. 2020, pp. 1–9. [Online]. Available: https://openreview.net/forum?id=XPZIaotutsD
[25] A. Grattafiori et al., "The llama 3 herd of models," 2024, *arXiv:2407.21783*.
[26] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," 2018, *arXiv:1804.07461*.
[27] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," 2016, *arXiv:1606.05250*.
[28] E. F. Tjong Kim Sang and F. De Meulder, "Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition," 2003, *arXiv:cs/0306050*.
[29] A. Imteaj, U. Thakker, S. Wang, J. Li, and M. H. Amini, "A survey on federated learning for resource-constrained IoT devices," *IEEE Internet Things J.*, vol. 9, no. 1, pp. 1–24, Jan. 2022.
[30] T. Nishio and R. Yonetani, "Client selection for federated learning with heterogeneous resources in mobile edge," in *Proc. IEEE Int. Conf. Commun.*, May 2019, pp. 1–7.
[31] X. Lisa Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," 2021, *arXiv:2101.00190*.
[32] S. Babakniya et al., "SLoRA: Federated parameter efficient fine-tuning of language models," 2023, *arXiv:2308.06522*.
[33] L. Yi, H. Yu, G. Wang, X. Liu, and X. Li, "PFedLoRA: Model-heterogeneous personalized federated learning with LoRa tuning," 2023, *arXiv:2310.13283*.
[34] J. Bian, L. Wang, L. Zhang, and J. Xu, "LoRA-FAIR: Federated LoRa fine-tuning with aggregation and initialization refinement," 2024, *arXiv:2411.14961*.
[35] J. Baxter, "A model of inductive bias learning," *J. Artif. Intell. Res.*, vol. 12, pp. 149–198, Mar. 2000.
[36] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of FedAvg on non-IID data," 2019, *arXiv:1907.02189*.
[37] Z. Allen-Zhu, "Natasha 2: Faster non-convex optimization than SGD," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, Dec. 2018, pp. 2675–2686.
[38] W. Ning et al., "Following the correct direction: Renovating sparsified SGD towards global optimization in distributed edge learning," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 2, pp. 499–514, Feb. 2022.
[39] D. Chen, L. Yao, D. Gao, B. Ding, and Y. Li, "Efficient personalized federated learning via sparse model-adaptation," in *Proc. Int. Conf. Mach. Learn.*, Jan. 2023, pp. 5234–5256.
[40] A. Shamsian, A. Navon, E. Fetaya, and G. Chechik, "Personalized federated learning using hypernetworks," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 9489–9502.
[41] E. Kreyszig, *Introductory Functional Analysis With Applications*, vol. 17. Hoboken, NJ, USA: Wiley, 1991.
[42] A. Tversky and I. Gati, "Similarity, separability, and the triangle inequality," *Psychol. Rev.*, vol. 89, no. 2, pp. 123–154, 1982.
[43] X. Li and P. Li, "Analysis of error feedback in federated non-convex optimization with biased compression: Fast convergence and partial participation," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 19638–19688.
[44] J. Wu, W. Huang, J. Huang, and T. Zhang, "Error compensated quantized SGD and its applications to large-scale distributed optimization," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, vol. 80, Jul. 2018, pp. 5325–5333.
[45] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2017, *arXiv:1711.05101*.
[46] D. Chen et al., "Data-juicer: A one-stop data processing system for large language models," in *Proc. Companion Int. Conf. Manage. Data*, Jun. 2024, pp. 120–134.
[47] Y. Wang et al., "Super-NaturalInstructions: Generalization via declarative instructions on 1600+ NLP tasks," 2022, *arXiv:2204.07705*.

Wanyi Ning received the B.E. degree from Beijing University of Posts and Telecommunications, Beijing, China, in 2020, where she is currently pursuing the Ph.D. degree in computer science and technology.

In 2023, she was a Visiting Student at ETH Zurich, Zürich, Switzerland. Her research interests focus on deep learning, federated learning, and distributed computing.

Jingyu Wang (Senior Member, IEEE) received the Ph.D. degree from Beijing University of Posts and Telecommunications, Beijing, China, in 2008.

He is currently a Professor with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications. His research interests include broad aspects of intelligent networks, edge/cloud computing, machine learning, AIOps, the IoV/IoT, SDN/NFV, knowledge-defined networking, and intent-based networking.

Dr. Wang is a Senior Member of the China Communication Society. He is selected for the Beijing Young Talents Program.

Qi Qi (Senior Member, IEEE) received the Ph.D. degree from Beijing University of Posts and Telecommunications, Beijing, China, in 2010.

She is currently an Associate Professor with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications. She has published more than 30 articles in international journals. Her research interests include edge computing, mobile cloud computing, the Internet of Things, ubiquitous services, deep learning, and deep reinforcement learning.

Dr. Qi was a recipient of two National Natural Science Foundation of China.

Haifeng Sun (Senior Member, IEEE) received the Ph.D. degree from Beijing University of Posts and Telecommunications, Beijing, China, in 2017.

He is currently an Associate Professor with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications. His research interests include AI, NLP, big data analysis, object detection, deep learning, and pattern recognition.

Daixuan Cheng received the bachelor's and master's degrees from Beijing University of Posts and Telecommunications, Beijing, China, in 2020 and 2023, respectively.

Her research interests include natural language processing, large language models, and domain adaptation.

Cong Liu received the Ph.D. degree from Beijing University of Posts and Telecommunications, Beijing, China, in 2008.

He is the Deputy Director of the Artificial Intelligence and Intelligent Operation Center, China Mobile Research Institute, Beijing, where he is the Project Manager. His research interests include AIOps and AI-enabled networks.

Lei Zhang received the master's degree from Nanjing University of Posts and Telecommunications, Nanjing, China, in 2004.

She is a Technical Expert of the Department of Cloud Network Center, China Unicom, Beijing, China, and the Leader of Digital Twin Project. Her research interests include mobile networks, network management, AIOps, and digital twin.

Zirui Zhuang (Member, IEEE) received the Ph.D. degree from Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2020.

From 2021 to 2022, he worked as a Post-Doctoral Researcher with BUPT. He is currently an Associate Researcher with the State Key Laboratory of Networking and Switching Technology and the School of Computer Science, BUPT. In 2019, he visited the Department of Electrical and Computer Engineering, University of Houston, Houston, TX, USA. His research interests involve network routing and management for next-generation network infrastructures, using machine learning and artificial intelligence techniques, including deep learning, reinforcement learning, graph representation, multiagent systems, and Lyapunov-based optimization.

Jianxin Liao received the Ph.D. degree from the University of Electronics Science and Technology of China, Chengdu, China, in 1996.

He is currently the Dean of the Network Intelligence Research Center and a Full Professor of the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing, China. He has published hundreds of research papers and several books. His research interests include cloud computing, mobile intelligent networks, service network intelligence, networking architectures and protocols, and multimedia communication.

Dr. Liao has won a number of prizes in China for his research achievements, which include the Premier's Award of Distinguished Young Scientists from National Natural Science Foundation of China in 2005 and the Specially-Invited Professor of the "Yangtse River Scholar Award Program" by the Ministry of Education in 2009.